University of Waterloo
Faculty of Engineering
Department of Electrical and Computer Engineering

Distributed Web Page Snapshotter

Group Number: 2015.007

Prepared by

(Names removed)

Consultant: Wojciech Golab
3 July 2014

This is a strong Detailed Design report. Strengths include the description of the final design, the use of upper-year knowledge, and some good quantitative analysis (see Section 3.3).

Table of Contents

List of Figures

List of Tables

1.0 High-Level Description of Project

1.1 Motivation

There are over 4.5 billion web pages as of June 2014 [1]. Looking at any point into the future, some web pages will no longer exist, or will change significantly. In most cases, there is no way of retrieving an older version of a web page after the web server changes the page. There should be an easy solution to save and then find deleted web pages. We hope to archive the web with enough coverage for historians, researchers and the public to look back at in the future.

1.2 Problem Statement

The quantity, quantity growth rate, and size of web pages are continuously growing [2]. It becomes a challenge for one source to keep up with the scale of web pages. These problems are already evident in existing solutions because they cannot cope with the amount of information shared on the web. Even the largest web archiving organizations are facing scaling issues. This means that the percentage of the archived web pages in the future will be getting smaller if we continue to use the current methods.

The objective of this project is to handle the archiving of the increasing amount of web pages without it being a monumental task when working with limited computing resources. The specific goal is to identify and take snapshots of web pages that are lesser known, short-lived, etc. instead of only the popular sites. Since the people who use the Internet would be most interested in expired web pages, we would like to incorporate them as a helping force for the solution of this problem.

1.3 Block Diagram

Figure 1 below shows the high-level architecture of the Web Page Snapshotter. The details are of components are explained the in following subsections. Blocks (subsystems) marked with a D are designed for our project, and blocks marked with a ND are not designed for our project.

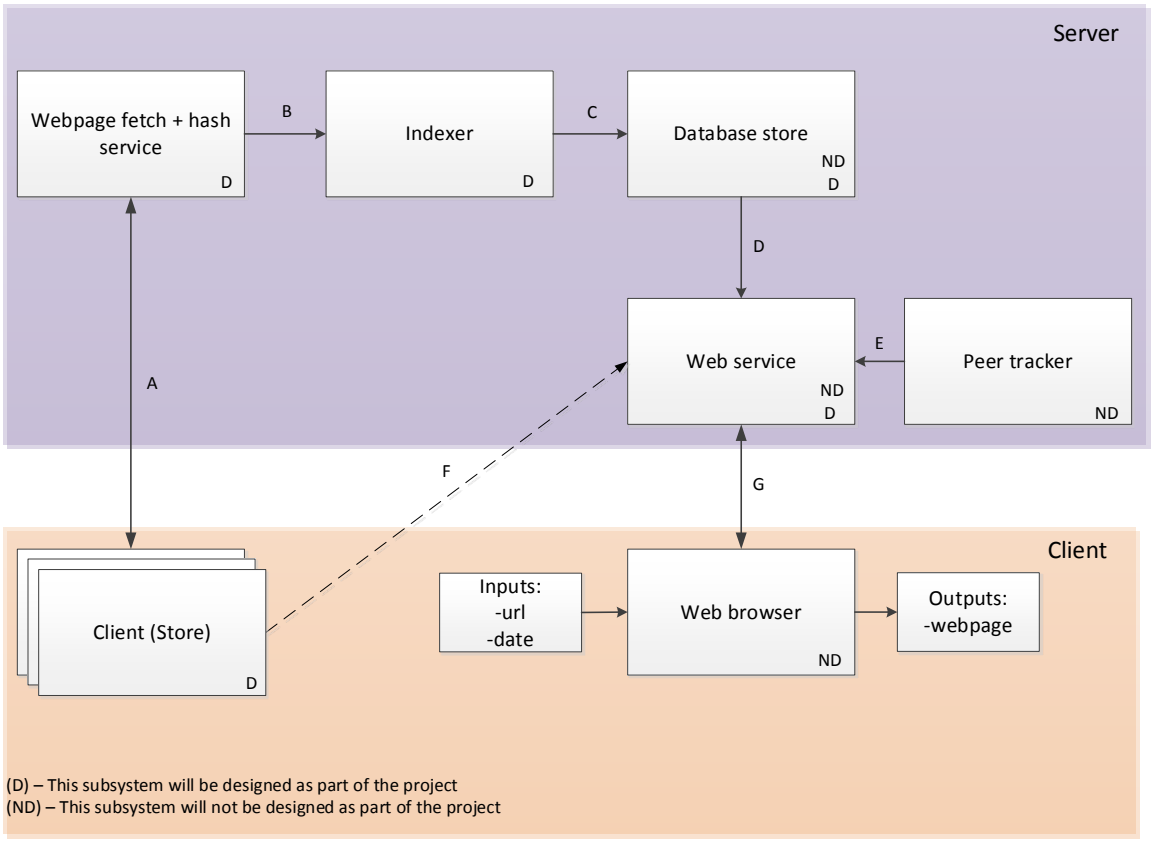


Figure 1: High-Level Block Diagram of Web Page Snapshotter Architecture

1.3.1 Description of the System Interactions

This section describes the system interactions outlined by the directed arrows in Figure 1.

A - The client stores the web page given by the web page fetch service. In addition, the hashing service is used to verify that the web page content is authentic.

B, C - The metadata for a web page snapshot is indexed and stored on a database.

D, E, F - The web service gets the metadata from the database and retrieves the web pages from the client(s) using the peer tracker.

G - The user can view the snapshots through a web browser by using a uniform resource locator (URL) and date as inputs. Our application will output a web page that is viewable by the user.

1.3.2 Breakdown of Designing and Not Designing Components

This section discusses the components in Figure 1 that we will design (D) and will not design (ND).

1.3.2.1 Server side

Web page fetch + hash service:

D - We will be designing a component that parses and replaces URLs from the web pages so that they point to local resources that are also archived by our project.

Indexer & Database:

D - We will be designing the database schema and indexing strategy for our design project.

ND - We will not be designing the database software that we are using. We will use an existing no-cost solution.

Peer tracker:

ND – We will be using the BitTorrent protocol to keep track of peers.

Web service:

D - Designing a website and application programming interfaces (APIs) to handle the backend requests (such as a uploading a web page snapshot, or a web page search result).

ND - Not designing the actual web server software. We will use an existing no-cost framework.

1.3.2.2 Client Side

Client (Store):

D - We will be designing an application or service that manages locally stored web pages that the client is responsible for. This allows the clients to create the torrent and seed it.

Client (web browser):

ND - We will not be designing this component. The user can just use a web browser of their choice to search for web pages.

2.0 Project Specifications

2.1 Functional Specifications

Table 1 below outlines the functional specifications of the Distributed Web Page Snapshotter.

Table 1: Functional Specifications of the Distributed Web Page Snapshotter

Specification	Classification	Description
Users are able to save web pages	Essential	A user is able to initiate the process of archiving a webpage. We need to identify which files to save for a web page snapshot because these files are shared with users. This also includes any required processing to get a functioning snapshot.
The user is able to search all web pages stored across all active clients	Essential	Our system intends to allow our users a way to share stored web pages with other users. Stored web pages are searchable by any other user. In order to implement searching for a web page, we must implement a search algorithm that utilizes a hash index on our database to search through all content available.
The web page must be fetched in 1 minute	Essential	The client application should have a hard limit of 1 minute to fetch data from the server. There are many databases that scale well with a billion records. Then the speed for downloading through BitTorrent depends on the number of peers and combined quality of connections.
Users are able to view the web page snapshots	Essential	The user application should keep track of the user's web page snapshots, and downloaded web page snapshots. The user can view any of the snapshots.
Details of a web page snapshot are uploaded to the server by the user application	Essential	We need to identify what details and metadata of a snapshot to uniquely identify the web page and date. This data is uploaded by the user application to the server.

The indexer and database supports the increasing number of web pages	Non-essential	The indexer is smart enough to uniquely identify web pages and date. The database software supports hash indexing, or has a very fast look-up design. The scale of data can easily be in the billions because there are billions of web pages where multiple snapshots will exist for each web page. This requirement is not essential because our project could still be functional with a slow indexing strategy.
The web service does not distribute exclusive peer trackers for the same web page	Non-essential	The exact same web page should not be shared exclusively between multiple peer trackers. This will help with performance when sharing and eliminate redundancy.
The client viewer is implemented as a web service	Essential	The client application is an application that runs on the client's computer. The web service provides the interface for all functionality features.
The client application should be supported on Windows	Essential	The operating system that the client application will support for the design project is the Windows Operating system.

2.2 Non-functional Specifications

Table 2 below outlines the non-functional specifications of the Distributed Web Page Snapshotter.

Table 2: Non-functional Specifications of the Distributed Web Page Snapshotter

Specification	Classification	Description
The user application must be user-friendly	Essential	An easy to use application means users are more likely to create web page snapshots. It is user-friendly because it has a simple interface that functions with minimal clicking/navigating.
The peer-to-peer aspect must be abstracted from the end user	Non-essential	The user should not have to worry about any of the peer-to-peer details such as seeding files or looking up in a distributed hash table for web pages.
The application should support tracking usage data and analytics for improvements	Non-essential	As an additional feature our application should store the user's browsing searching habits to better improve the application in the future.
The hosted files should go through server validation to check for authenticity	Non-essential	The hosted web pages must be validated before they can be viewed by end users. This is to protect our application from spam and malicious files being shared between users.

3.0 Detailed Design

3.1 Web Page Fetch & Hash Service Design

This component of the system is responsible for:

- Fetching the webpage and all its hard resources
- Replacing the links for all resources with ones that link to the same domain
- Packaging the resources into an archive file and distributing it to the user
- Making an API call to the indexing service once the archive starts being seeded

These four steps can be broken up into two different phases: the fetch and replace phase, and the package distribution phase.

3.1.1 Fetch and Link Replacement Phase Design

The goal of the fetch and link replacement phase is to retrieve a specified URL's webpage, replace all links (hyperlink), download all associated files, and then package them into one archive file. This archive file contains the web page, along with all images, JavaScript files, CSS files, and any other resources that is needed to display the web page properly. Our project uses the Zip archive file format.

3.1.2 Link Replacement Design

There are five different HTML tags where links can appear in a web page. For each of these tags, the links are modified/replaced to point to local pages and resources. This is important, because when the viewer application displays the page, all resources (images, JavaScript files, CSS files, etc.), and links to other pages (on the same domain or another domain) must point to copies of those resources. If the snapshots pointed to the original links, visiting the links could bring the user to a completely different website, or to an outdated version of the page or resource.

Table 3 on the next page lists the five tags, the attribute in which the link appears in, and the type of replacement that must be made. A hard resource is a resource that must be downloaded by the server, and then packaged into the archive file (such as an image). A soft resource is a link to

another page and does not need to be packaged, but still needs to be replaced to point to a local copy.

Table 3: Web Page Snapshot Link Replacement Breakdown

HTML Tag	Attribute	Resource Type (Replacement Type)
<img>	src	Hard resource
<script>	src	Hard resource
<link>	href	Hard resource
<a>	href	Soft resource
<iframe>	href	Soft resource

All links must be replaced according the following rules:

If the link points to a **hard resource**, then replace it with a new link of the format:

/r/{hash}.{extension}?ct={content_type}

Where,

- {hash} is the SHA-1 hash of the resource (this ensures unique resource names)
- {extension} is the file extension of the resource
- {content_type} is the URL-encoded content type of the resource returned in the response header when it was originally requested

If the link points to a **soft resource**, then replace it with a new link of the format:

/w/?url={expanded_url}&time={timestamp}

Where,

- {expanded_url} is the URL-encoded complete URL of the original resource
- {timestamp} is the UTC timestamp, in seconds, of the current time

The abstracted pseudocode for the link replacement and packaging process is as follows:

```
# Download the specified web page and all its resources, and then zip it up in
an archive
def replace_and_package_webpage(url):
    # Download the html content at the specified url
    html_page = get_html_page(url)

    # Process each tag in the page that contains a link
    for tag in html_page:
        if tag_contains_link(tag):
            if is_a_hard_resource(tag):
                downloaded_resources += download_file(tag.url)
            rewrite_tag(tag)

    # Zip up all the downloaded files and the html file
    file = zip(html_page, downloaded_resources)
    return file
```

3.1.3 Package Distribution Phase Design

The goal of the package distribution phase is to distribute the archive file generated in the fetch and replace phase. After the archive file is passed to the user, they can start seeding the archive, and therefore allow the snapshot to be available to other users.

In order to distribute and begin sharing web page snapshots, three things must be done:

1) The archive file must be sent to the user
2) The user must start seeding the archive file
3) The server must index the torrent file or magnet link of the web page snapshot

At this point, the web page snapshot is searchable by everyone, and can be download and viewed by sharing the archive file. For this process, two possible design alternatives were considered:

1. The archive file is downloaded via standard Hypertext Transfer Protocol (HTTP). At this point, the user is responsible for manually creating the torrent and begin seeding it. Once the torrent has been created, the magnet link or torrent file is uploaded to the server in a secondary request, which will index the archive file though the indexing service. Alternatively, the downloading, seeding, and secondary API request can be done automatically though a custom BitTorrent client, written by ourselves, that runs on the user's machine.

The primary advantage of this approach is that it is simple to implement. The only overhead (on the server side) is the state information that must be saved between the initial archiving request, and the request to index the associated torrent file. However, a custom BitTorrent client must be written and run on the end user's computer. The necessity to run this program would act as a friction between the user and the process of archiving web pages. However, the automated process is far better than needing the user to handle files manually.

2. The torrent is created and initially seeded by the fetching service itself. In this case, the associated torrent file is given to the user, who then must download and seed the archive using any third party torrent client. Since the server is handling the initial seeding, it can automatically index the torrent file itself. It should stop seeding the archive when the user has downloaded the archive file via BitTorrent.

   The primary advantage of this design is that the user does not need to worry at all about the archive file. All the client needs to do is download the torrent file and start seeding it. It is essentially the least frictional process that is available. However, this complicates the design of the server, and it requires a lot more resources (initial seeding) than the first design. This can be a problem when it comes around to scaling the service.

Both of these designs satisfies the relevant functional and non-functional specifications for this project. Most importantly, the complexity of the fetching, seeding, and indexing can be hidden from the user, and the underlying distribution mechanism abstracted. This allows for the process of indexing, and seeding new web pages to be quick and easy for the user.

From these two alternatives, the first design was chosen. This is because the simplicity of design that comes with handling the initial seeding of the archive on the user's side, rather than dealing with the complexities of keeping track of the initial seeding ourselves creates a cleaner and more robust design.

3.1.4 Technology stack

The technology stack that is used to implement the web page fetch and hash service is shown in Table 4 on the next page. We choose to stick to a Python implementation for our backend services because we have experience developing using Python.

Table 4: Web Page Fetch and Hash Service Technology Stack Breakdown

Component	Technology	Reason
Web server framework	Flask (Python)	Lightweight and easy to use
Html scraper library	BeautifulSoup (Python)	Complete, feature-filled, easy to use
Distributed memory store	Redis	Fast, open-source

3.2 Database and Indexer Design

The design of the database and indexer components are closely related to each other, so the detailed description of both are featured in this section.

These two components are responsible for indexing and storing metadata of archived resources. They act as the central component that connects the web page fetch service (which will do a majority of the indexing), and the web service (which performs queries against the database for archived resources).

Many aspects need to be considered when designing these components because they interact with multiple pieces of the project. Good design and implementation is crucial for a reliable service with fast access to information on web page snapshots.

These aspects include:

- What information is stored so that the web server is able to and has enough information to retrieve the archived pages
- How the database schema is indexed to provide fast look-up
- What constraints are enforced to eliminate ambiguity and ensure the database is always in a consistent state
- How other services interact with these components
- How failures are handled to prevent downtime

With these aspects in mind, we can now evaluate the feasibility of different options.

3.2.1 Database Stored Content Design

The first aspect that needs to be looked at is what data has to be stored. Ideally, we would want to store the least amount of information that can still allow us to find the desired resources.

Since the most common way of collaboratively sharing content over the BitTorrent protocol is through the use of torrent files and magnet links, we store either the torrent files or the magnet links. However, this solution surfaces some issues. For instance, how the page is packaged into a torrent file will affect what needs to be stored. For this, we consider two alternatives:

1. Package each resource within a web page (images, scripts, videos, etc.) separately into different torrent files. This ensures that resources that are identical across different domains can be shared, but this creates a huge amount of metadata that we need to store. Another disadvantage of this technique is that it makes the retrieval process less reliable, since all the individual pieces are tracked separately.
2. Package all resources within a web page into a single torrent file. This significantly reduces the amount of data that needs to be stored for a snapshot, since each page is a single entry within the database. This separates identical resources by domain, which means that there may be less seeders for popular resources. However, this method requires less tracking, since everything is packaged into a single torrent file.

We decided to use the second alternative since it requires less data to be stored, is easier to manage, and is more reliable. It also means that we can use a NoSQL database like MongoDB since the data model is very simple. The schema for the data stored is:

```
{
   _id: <string>,
   url: <string>,
   info_hash: <string>,
   timestamp: <i64>
}
```

Where:

_id is a unique document ID
url is the URL of the web page
info_hash is the BitTorrent Info Hash
timestamp is when the web page was archived

The unique document ID is included to support future optimizations such as caching specific archives. We will also be sharding our database store as the amount of data increases to satisfy the functional specification of the indexer and database supporting an increasing number of web page snapshots.

3.2.2 Indexing Strategy

A typical use case for a search query is finding an archived version of a URL that is near a given timestamp. For this, we would need to ensure that the URL and timestamp fields are indexed. Since all indexes in MongoDB are B-tree indexes, both range and equality queries would be very efficient and satisfy the design specification that the indexer supports an increasing number of web pages.

We also need to ensure that the query results are consistent. Therefore, we need to put a unique constraint on the index, which is, no documents can have the same URL and the same timestamp. This is because two documents sharing the same URL and timestamp can have different hashes because multiple requests made at the same time to the fetching service can result in different hashes due to varying dynamic content on the web page. In this case, we cannot produce a result that is guaranteed to be the same unless there is only one possible result.

3.2.3 Database Interface Design

The interface dictates how the component is used by other services. For this aspect of the component, we have considered more than two alternatives. Thus, we use a decision matrix to weigh the different criteria and compare the scores of each alternative. The decision matrix, with criteria weights shown in the brackets, is shown in Table 5 on the next page. The criterion of scalability is the most valuable in for this project because there is a large number of potential services we may want to support in our project in the future. Ease of implementation is given the lowest weight because as fourth-year engineering students we should be able to implement a solution without being concerned of its level of difficulty.

Apache Thrift is a framework for building services. It includes a code generation engine that can generate server and client code in any language given a schema that describes the interface. Since we can launch the service independently, it is highly scalable in a sense that we can have machines solely used for this service. It is also relatively easy to implement because all that is

needed is the schema. Once the schema is written, client code can be generated based on the language of which the other components are written, so it is also relatively easy to use.

Another alternative is using a web service, which provides the same level of scalability because it can be launched independently. It is the most difficult to implement because a web server needs to configured and extra design work is required to handle different query parameters. However, once it is implemented, it is very easy to use since any client can initiate simple HTTP requests.

We can also write a library in a particular language that can be used by other components. However, if we choose to only do this, then it has to be packaged into another component, which is not as scalable. It is very easy to implement, but since it is language-specific, it is not as easy to use if the other components are written in a different language.

Table 5: Database Interface Decision Matrix

	Scalability (0.5)	**Ease of Implementation (0.2)**	**Ease of Use (0.3)**	**Total**
Apache Thrift	90%	80%	95%	89.5%
Web service	90%	60%	100%	87%
Library	50%	100%	60%	63%

After considering each alternative in all the criteria, Apache Thrift scores the highest out of all the alternatives. Therefore, it will be used for the interface.

To handle failures and prevent downtime MongoDB recommends the use of replica sets as the replication strategy. It provides high availability because multiple copies of the data are stored in different machines. When a secondary member in the set fails, other members will take over and when a primary fails, an election takes place which chooses a new primary.

3.3 Website Design

The website or front-end component serves the purpose of being able to search the available web page snapshots and display the results. The front-end is the main entry point of the service and the primary component used directly by the users. The component can be divided into two sections the HTML/JavaScript in the browser client side and the server side.

The client side runs in a browser and is responsible for making requests to the server side for web page snapshots with the desired URL and dates, and then displaying the results in the browser. This part of the component is fairly lightweight and simple, needing only to make simple web requests so the jQuery library is used since it is lightweight and easy to use. All members have experience with jQuery, and since its usage is not substantial, no alternatives will be considered.

For the server side, it is responsible for taking requests from the user and connecting it to the BitTorrent component to find the requested archived web page package. The component needs to then take the package, decompress it and then serve the files to the user. The design decisions that need to be made are which web server to use, and which web framework to use.

The web server is the software component that listens for requests on a port, such as port 80 (HTTP), queues requests, and then passes it to the web framework. The web framework is a set of APIs that aids in the handling of HTTP requests by user code. The selection of the web server depends on the time performance of the web server and web framework to handle requests. The performance of this component is important because it is a significant part of the main flow of the service, so any delay will worsen the responsiveness of the service. The performance also includes the ability to handle multiple requests simultaneously. Another important factor for the selection of the web server and web framework is the ease of use and deployment.

Due to the lightweight nature of the service, only lightweight servers and frameworks are selected for comparison. The web servers compared are Tornado, Gunicorn, Twisted, Flask and CherryPy. The web frameworks compared are Tornado, Flask and CherryPy. Some software names are listed twice because those packages contain both a web server and web framework. Most Python web frameworks make use of the Web Server Gateway Interface (WSGI) interface to easily mix combinations of web servers and web frameworks to test.

Tornado is a non-WSGI compliant open source non-blocking web server and web framework package written completely in Python that is designed for high performance. Gunicorn is a lightweight WSGI compliant web server developed from the Ruby Unicorn package, with support for multiple workers, and is run with 4 worker threads. Twisted is an event driven web server and framework with support for WSGI web applications. Flask is a lightweight WSGI compliant web framework that is designed for simple development. CherryPy is another WSGI compliant web framework designed for rapid development.

Due to the WSGI compliance, it is easy to combine web servers and web frameworks in many different combinations. The web server and web framework combinations being compared are:

- Flask(Werkzug)/Flask
- Gunicorn/Flask
- Twisted/Flask
- Tornado/Flask
- CherryPy/CherryPy
- Gunicorn/CherryPy
- Tornado/CherryPy
- Twisted/CherryPy
- Tornado/Tornado

The tests that will be used to compare the performance of the combinations are the time to perform a single simple request, the time to perform a single request involving a database (DB) query, the time to perform simultaneous requests involving database queries. The database queries being used are somewhat randomized in order to prevent any caching being done on the database side. The final test is to test simultaneous calls to a request that will block in order to test if the system is asynchronous (this is being tested by having the request sleep for 0.5 seconds). The test results for the performance of the combinations of web servers and web frameworks are shown in Table 6 on the following page. The relative scores for performance in comparison to the slowest result are shown in Table 7 with the weighting of the type of query contained in brackets.

Table 6: Raw Performance Result Times of Web Server and Web Framework Combinations

Web Server	Web Framework	Single Request (ms)	Single DB query Request (s)	Multiple DB Query Requests (s)	Blocking Query (s)
Flask	Flask	2.2110	0.2451	7.09723	10.0382
Gunicorn	Flask	3.0648	0.251982	8.62686	2.5246
Twisted	Flask	2.6109	0.234025	8.9832	0.545735
Tornado	Flask	3.9098	0.2691	8.93211	10.038197
CherryPy	CherryPy	3.9098	0.2691	10.627421	10.038197
Gunicorn	CherryPy	3.0841	0.249421	7.13062	2.538177
Twisted	CherryPy	4.0440	0.387403	8.85697	0.549896
Tornado	CherryPy	2.4628	0.261076	9.095822	10.052501
Tornado	Tornado	6.1569	0.243632	9.326162	0.535342

Table 7: Relative Scores of Table 6 for Performance

Web Server	Web Framework	Single Request (5%)	Single DB Query Request (5%)	Multiple DB Query Requests (10%)	Blocking Query (80%)	Total
Flask	Flask	0.6408	0.3673	0.3321	0.0014	0.0847
Gunicorn	Flask	0.5022	0.3495	0.1882	0.7488	0.6604
Twisted	Flask	0.5759	0.3959	0.1547	0.9457	0.8206
Tornado	Flask	0.3649	0.3053	0.1595	0.0014	0.0506
CherryPy	CherryPy	0.3649	0.3053	0	0.0014	0.0346
Gunicorn	CherryPy	0.4990	0.3561	0.3290	0.7475	0.6736
Twisted	CherryPy	0.3431	0	0.1665	0.9452	0.7900
Tornado	CherryPy	0.5999	0.3260	0.1441	0	0.0607
Tornado	Tornado	0	0.3711	0.1224	0.9467	0.7881

From looking at the results, every combination has relatively close performance values for the single request, single DB query request and multiple DB query request. Due to the fact that the actual performance penalty is from the overhead of the network request of the system and not the handling of the request. The blocking request test is the time to process 20 simultaneous requests

where each request sleeps the thread for 0.5 seconds – so the systems that take 10 seconds to process all requests are not asynchronous at all, and the systems that take 0.5 seconds are fully asynchronous. The systems that take 5 seconds use a limit of 4 asynchronous processes running simultaneously.

Since this system is intended to be used by multiple people simultaneously, an asynchronous system is the most important. The other aspects of performance are equally as important are not much different, so they are ignored. The ease of use of the system depends on the choice of web framework and web server. The Tornado framework is the most difficult to use since, unlike the others, it is not WSGI compliant. Flask is the easiest framework to use, CherryPy slight less, and Gunicorn and Twisted being equally easy to use web servers.

Table 8: Decision Matrix Comparison of Web Servers and Web Frameworks

Web Server	**Web Framework**	**Performance (85%)**	**Simplicity (15%)**	**Total**
Flask	Flask	0.08476	0.85	0.19955
Gunicorn	Flask	0.66049	0.85	0.68892
Twisted	Flask	0.82063	0.85	0.82503
Tornado	Flask	0.05060	0.8	0.16301
CherryPy	CherryPy	0.03465	0.8	0.14945
Gunicorn	CherryPy	0.67367	0.8	0.69262
Twisted	CherryPy	0.79005	0.8	0.79154
Tornado	CherryPy	0.06071	0.75	0.16410
Tornado	Tornado	0.78819	0.5	0.74496

According to the decision matrix shown in Table 8 above, the best combination is the Twisted and Flask combination. This is due to the full asynchronous support and the ease of use. We will use Twisted and Flask for our project.

3.4 Client (Store) Application Design

The client (store) application is a Windows application that manages web page snapshots on the user's computer and provides the functionality for users to host web pages using the BitTorrent protocol. Our application only needs to run on the Windows operating system per our functional specifications.

3.4.1 User Interface

The client application has a small set of screens to view and manage web page snapshots in a list format. The user is responsible for organizing their web page snapshots anywhere on their computer because the application does not dedicate one location for snapshots. The user interface will have minimal number of interaction elements to keep it simple: add a snapshot to application, remove a snapshot, a checkbox for each snapshot to permit or deny hosting the file (seeding). There is a panel to see the BitTorrent sharing statistics such as which files are currently being seeded and network traffic. This simple and easy-to-use interface, as explained previously, is to meet our goal of having a user-friendly application.

3.4.2 Type of Desktop Application

The client can be supported in Windows as either a web application or a desktop application. A desktop application is preferred because we do not have to deal with the troubles of compatibility with many web browsers and versions. Also, our own desktop application allows us to manage versions and the process to access required resources: file system for managing web page snapshots, and network interfaces and protocols for BitTorrent.

The next step is to determine which programming language and framework to use for developing the application. The options compared for programming languages and frameworks – or referred to as toolkit – were narrowed down to Java (Swing), Python (Tkinter), and C# Windows Presentation Foundation (WPF). The criteria used to evaluate the application are previous knowledge, performance, developer support, ease of implementation, and portability.

Table 9 on the following page shows the weighting schemes and explanation of their weights. Previous knowledge is a low weight because we are fourth-year students who are experienced enough to learn and use new programming languages and frameworks without significant delay.

Performance is always a big concern because it is ideal if users leave the application running all the time in order to seed – so we must minimize resource usage – users also should be able to access web page snapshots quickly. Developer support refers to the amount of improvement effort that goes into the framework by the developer community or cooperation. This is important to ensure that new functionality may be supported by our application as the framework grows. The ease of implementation is important to complete the project in the given deadline – we want to focus on design and functionality related to our project idea instead of the dealing with the framework. Portability is important to expand our application to be available operating systems.

Table 9: Weight Scale of Criteria Used for Comparing Framework

Criterion:	**Previous Knowledge**	**Performance**	**Developer Support**	**Ease of Implementation**	**Portability**
Weight:	10%	25%	20%	25%	20%

Each graphical user interface (GUI) toolkit is compared against each other based on online research and our own judgement. A score is given for each toolkit for each criteria is shown in in a decision matrix format in Table 10 below. Our group members have not developed GUIs with any of the toolkits, unless otherwise stated, meaning previous knowledge will tend be low.

The Swing toolkit utilizes the Java programming language which several of our group members are comfortable with, giving it a middle score of 6. The performance of Swing components is slower compared to other toolkits and it has been reported to have video issues on various platforms so it receives a score of 6 for performance. Java Swing is maintained by Oracle for over a decade and counting [3] and has a lot of documentation, making it easy to pick-up and use. Swing is a supported well and Java runs on all standard desktop operating system platforms, meaning good portability. It is very common for users to have Java.

The Python toolkit Tkinter is included with the standard installation of Python, making it readily available for development. Several of us are familiar with Python, which gives it a score of 6. The performance of Tkinter is relatively decent. Since Tkinter is the most commonly used Python GUI toolkit by the Python community, it receives a score of 9 in terms of developer support [4]. Tkinter also features plenty of documentation and open libraries to make it easy to add a variety of application features. Finally, since Python, and Tkinter, is compatible with every operating system, it receives the best score in terms of portability. Python is not as widely installed as Java.

The third option is to use the C# with WPF which is the only GUI toolkit our group members have experience with, giving it a higher score than the other options. The performance of a WPF application was given a score of 5 because the way how the programming language is designed it is not meant to be highly responsive [5]. Since Microsoft has great documentation, and is actively developing the WPF toolkit, it has tremendous support in the future for potentially new features. Since WPF applications are restricted the Windows, it receives a very low score of 2 for portability.

Table 10: Decision Matrix Analysis of Frameworks for Desktop Application

	Previous Knowledge	**Performance**	**Developer Support**	**Ease of Use**	**Portability**	**Total**
Java (Swing)	6	8	8	8	9	8
Python (Tkinter)	7	7	9	8	9	8.05
C# (WPF)	8	8	10	8	2	7.2

The decision matrix results in Table 10, shows that the marginally best choice is the Python (Tkinter) toolkit. The toolkit has a strong developer community and meets the specification of being supported on Windows. Another reason why it is justified to use Python is because several of our team members are already familiar with the language, like it, and our web services are also using Python. This makes it easier for all team members to collaborate, combine, and understand different subsystems of the project when working with one programming language.

3.4.3 Conclusion

Tkinter is used to provide the user interface for our application. The design is done from a minimalistic perspective, and it is kept simple as described in section 3.6.1 to achieve the specification of being user-friendly. The technicalities of the BitTorrent protocol is not displayed on the application to meet the specification of abstracting out the peer-to-peer aspects. However, this data is voluntarily stored and transferred to server to allow user usage data and analytics for improvements.

Python has support for input and output, and in conjunction with configuration and data files, we can maintain the list and location of web page snapshots saved within the application. Our Python code uses Rasterbar's Software libtorrent [6] Python library to help with BitTorrent. The library provides the application programming interface for our application to handle snapshot seeding

through BitTorrent, working with torrent files, and working with magnet links. The application connects to the server via HTTP and sends the torrent file or magnet link. The file and data transfer with the help of libtorrent allows user applications to store web pages on their machine and upload them.

4.0 Discussion and Project Timeline

4.1 Discussion on Design

Table 11 below shows how many hours each student has worked so far on the project.

Table 11: Number of Hours Worked

Group Member	# of Hours
(Names removed)	70
	65
	63
	62
	60

Our project design, across all subsystems, utilizes a lot of upper year knowledge to complete. This includes knowledge from Distributed Systems (ECE454), Computer Networks (ECE358), Database Systems (ECE356), and Operating Systems (ECE254). Our design utilizes the BitTorrent protocol, which requires knowledge of distributed hash tables and the topology of peer-to-peer networks that we study in ECE454. The BitTorrent protocol also requires knowledge of computer sockets, and multicast routing schemes that we study in ECE358. The database system, schema, and design for our specification requires fast look-up and huge data store, which requires knowledge on normalized schemas and indexes from ECE356.

Overall, our design meets almost all of our original design specifications. Our design may not completely satisfy the specification that states, “The peer-to-peer aspect must be abstracted from the end user”. We try to use common terminologies and very straightforward use cases to share snapshots. However, our client application does display some information on snapshot sharing statistics done over BitTorrent, and has options to permit or deny snapshot sharing. The concept of snapshot files and sharing is easy to grasp for the application’s purpose.

Our design is novel because it does something that none of us has hard before – use the BitTorrent protocol to enable a distributed storage network of files that are shared between users. Any user can choose to save snapshots and lets them host the content that they care about. The users do not need to have much knowledge of the BitTorrent protocol and how it works. There are certain aspects of our design that are elegant, such as generation of magnet links with our

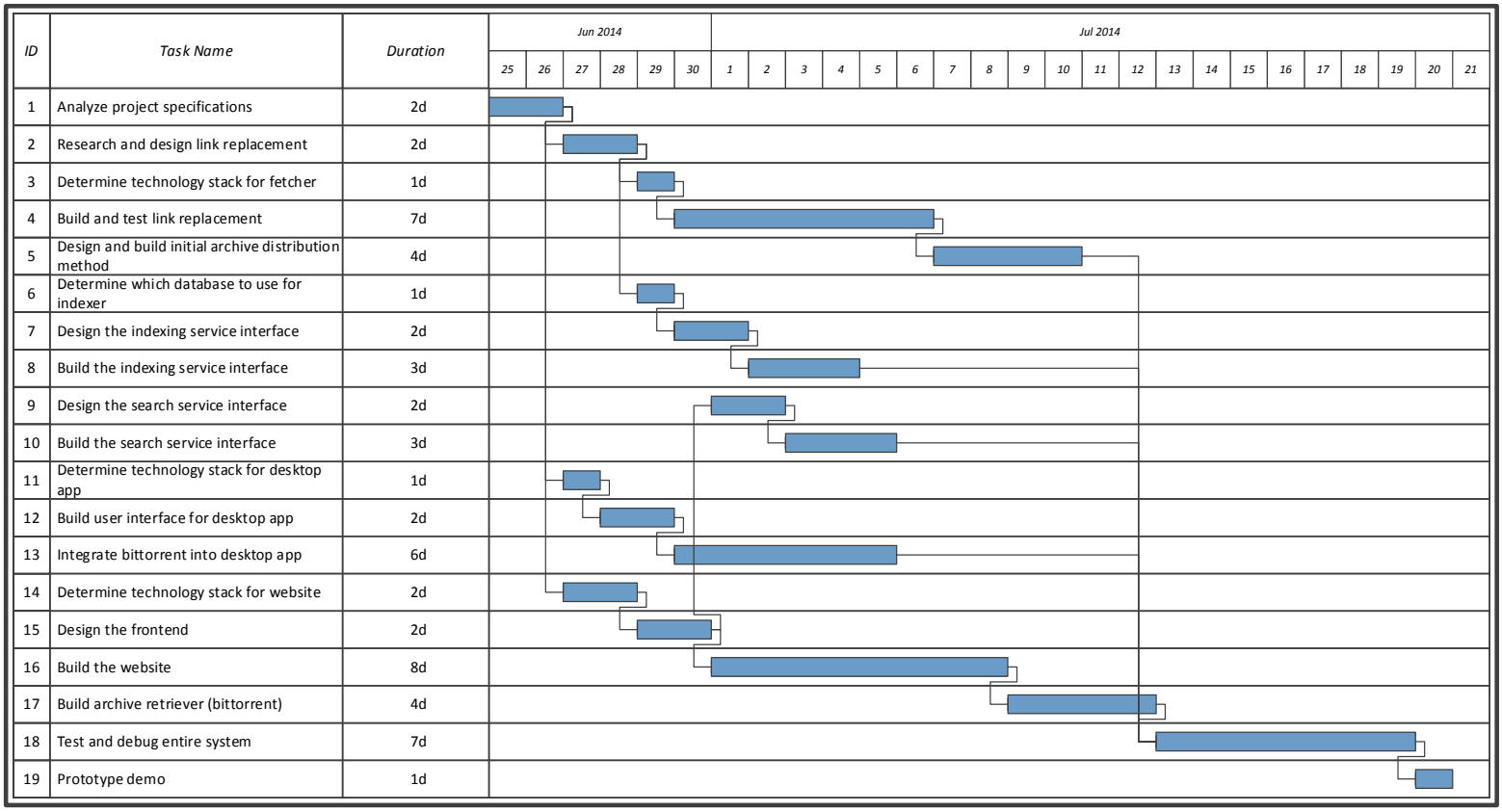
webpage fetch and hash service subsystem. This service helps users save web page snapshots that they want to host and share to others as easy as clicking a button.

We will probably need to refine several aspects while we design our project prototype to adapt to the web page fetch and hash strategy to work with the web service. Additionally, we expect to perform multiple design iterations on the client application's user interface to best the minimalistic design perspective from our perspectives.

Our project is a purely software project there are no potential safety hazards.

4.2 Project Timeline

The project timeline for our ECE498A term is displayed with task dependencies as a Gantt chart in Figure 2 below. Following the timeline we have come up with, we expect a working prototype to be completed and ready to demo by July 24, 2014 for our consultant.



ID	Task Name	Duration
1	Analyze project specifications	2d
2	Research and design link replacement	2d
3	Determine technology stack for fetcher	1d
4	Build and test link replacement	7d
5	Design and build initial archive distribution method	4d
6	Determine which database to use for indexer	1d
7	Design the indexing service interface	2d
8	Build the indexing service interface	3d
9	Design the search service interface	2d
10	Build the search service interface	3d
11	Determine technology stack for desktop app	1d
12	Build user interface for desktop app	2d
13	Integrate bittorrent into desktop app	6d
14	Determine technology stack for website	2d
15	Design the frontend	2d
16	Build the website	8d
17	Build archive retriever (bittorrent)	4d
18	Test and debug entire system	7d
19	Prototype demo	1d

Figure 2: Gantt Chart for ECE498A Project Timeline

The project timeline of our work for ECE498B is predicted to appear as follows in Figure 3 below. We do not have concrete tasks yet, but this timeline is a good approximation for what we expect our schedule to be.

ID	Task Name	Start	Finish	Duration
1	Analyse current design against project specifications	1/05/15	1/06/15	2d
2	Work on Desktop Application User Interface	1/11/15	2/09/15	30d
3	Work on website design and functionality	1/07/15	2/05/15	30d
4	Implement new backend services	1/15/15	2/03/15	20d
5	Final Report	2/01/15	2/15/15	15d
6	Full system integration testing	2/10/15	2/16/15	7d
7	Bug Fixes	2/21/15	3/07/15	15d
8	Final Prototype Demonstration	3/11/15	3/11/15	1d
9	Symposium Preparation	3/12/15	3/16/15	5d

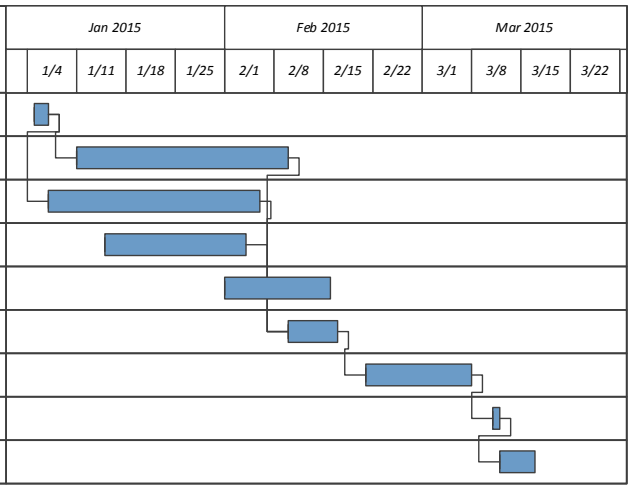


Figure 3: Gantt Chart for Estimated ECE498B Project Timeline

References

[1] M. Kunder, “WorldWideWebSize.com,” 01 Jun. 2014; http://www.worldwidewebsize.com/.

[2] A. Trotman, J. Zhang, “Future Web Growth and its Consequences for Web Search Architectures,” arXiv, 4 Jul. 2013; http://arxiv.org/abs/1307.1179v1.

[3] Wikipedia contributors, “Swing (Java),” 28 Jun. 2014; http://en.wikipedia.org/w/index.php?title=Swing_(Java)&oldid=596930677

[4] Wiki contributors, “TkInter,” 28 Jun. 2014; https://wiki.python.org/moin/TkInter

[5] Wikipedia contributors, “Windows Presentation Foundation,” 28 Jun. 2014; http://en.wikipedia.org/w/index.php?title=Windows_Presentation_Foundation&oldid=606414459

[6] Rasterbar Software, “Libtorrent,” 28 Jun. 2014; http://www.rasterbar.com/products/libtorrent/